

Convex Optimization

Homework 3.

$$1. \begin{cases} \min & \frac{1}{2} \|z\|_2^2 + \lambda \|w\|_1 \\ \text{s.t.} & z = Xw - y \end{cases}$$

The Lagrangian is:

$$L(w, z, \mu) = \frac{1}{2} \|z\|_2^2 + \lambda \|w\|_1 + \mu^T (z - Xw - y)$$

the dual function is

$$g(\mu) = \inf_{w, z} L(w, z, \mu) = \inf_z \left(\underbrace{\frac{1}{2} \|z\|_2^2 + \mu^T z}_{f(z)} \right) + \inf_w \left(\underbrace{\lambda \|w\|_1 - \mu^T Xw}_{Q(w)} \right) + \mu^T y$$

since f is differentiable $\nabla f(z) = 0 \Rightarrow \|z\|_2 + \mu = 0 \Rightarrow z = -\mu$

$$\inf_z f(z) = \frac{1}{2} \|\mu\|_2^2 + \mu^T \mu = -\frac{1}{2} \mu^T \mu$$

for $Q(w)$ we can use the result of exercise 2.1 from Homework 2 for $h(w) = \|w\|_1$

$$h^*(w) = \begin{cases} 0 & \text{if } \|w\|_\infty \leq 1 \\ \infty & \text{otherwise} \end{cases}$$

$$\inf_w (\lambda \|w\|_1 - \mu^T Xw) = -\sup_w |-\lambda \|w\|_1 + \mu^T Xw| = \lambda h^* \left(\frac{\mu^T X}{\lambda} \right)$$

$$\text{So } g(\mu) = -\frac{1}{2} \mu^T \mu + \lambda h^* \left(\frac{\mu^T X}{\lambda} \right) + \mu^T y$$

and the dual problem is

$$\begin{cases} \maximize & \mu^T y - \frac{1}{2} \mu^T \mu \\ \text{s.t.} & \|\mu^T X\|_\infty \leq \lambda \end{cases} \Leftrightarrow \begin{cases} \min & \frac{1}{2} \mu^T \mu - \mu^T y \\ \text{sup} & \|\mu^T X\|_\infty \leq \lambda \end{cases}$$

$$\|\mu^T X\|_\infty \leq \lambda \Leftrightarrow -\lambda \leq [\mu^T X]_i \leq \lambda \quad \forall i = 1, \dots, n$$

$$\Leftrightarrow [\mu^T X]_i \leq \lambda \text{ and } [-\mu^T X]_i \leq \lambda \quad \forall i = 1, \dots, n$$

$$\Leftrightarrow A\mu \leq \lambda 1_{2d} \quad \text{with } A = \begin{pmatrix} x^T \\ -x^T \end{pmatrix}$$

So we can write the problem as follows

$$\begin{cases} \min & \mu^T Q \mu + p^T \mu \\ \text{st} & A\mu \leq b \end{cases}$$

$$\text{with } Q = \frac{1}{2} I_n \geq 0$$

$$p = -y$$

$$A = \begin{pmatrix} x^T \\ -x^T \end{pmatrix}$$

$$b = \lambda 1_{2d}$$

2. Since μ is not equal to 0, the goal is to solve

$$\min (v^T Q v + p^T v) = \sum_{i=1}^n \log(b_i - (A v)_i)$$

$$\nabla g(v) = (2Qv + p) + \sum_{i=1}^n (b_i - a_i^T v)^{-1} a_i$$

$$\nabla^2 g(v) = 2Q + \sum_{i=1}^n (b_i - a_i^T v)^{-2} a_i a_i^T$$

In [41]:

```
import numpy as np
import matplotlib.pyplot as plt

def centering_step(Q, p, A, b, t, v0, eps):
    """
    Implements the Newton method to solve the centering step for a barrier method.

    Parameters:
        Q (ndarray): Quadratic term in the objective function (symmetric matrix).
        p (ndarray): Linear term in the objective function.
        A (ndarray): Constraint matrix.
        b (ndarray): Constraint vector.
        t (float): Barrier parameter.
        v0 (ndarray): Initial variable.
        eps (float): Target precision for stopping criterion.

    Returns:
        v_seq (list): Sequence of iterates (vi) during optimization.
    """
    v_seq = [v0.copy()] # Initialize sequence with v0

    while True:
        # Compute gradient
        grad = t * (2 * np.dot(Q, v0) + p) + np.dot(A.T, 1 / (b - np.dot(A, v0)))

        # Compute Hessian
        barrier_diag = np.diag(1 / (b - np.dot(A, v0))**2)
        hess = 2 * t * Q + np.dot(A.T, np.dot(barrier_diag, A))

        # Compute Newton step
        delta = -np.linalg.solve(hess, grad)

        # Compute Newton decrement
        lambda_sq = np.dot(grad.T, delta)

        # Stopping criterion
        if lambda_sq / 2 <= eps:
            break

        # Backtracking line search
        alpha, beta = 0.1, 0.5
        s = 1.0

        # Current barrier function value
        def barrier_func(v):
            return t * (np.dot(v.T, np.dot(Q, v)) + np.dot(p.T, v)) - np.sum(np.log(b - np.dot(A, v)))

        f_current = barrier_func(v0)

        while True:
            v_new = v0 + s * delta

            # Check interior points constraint
            if np.all(b - np.dot(A, v_new) > 0):
                f_new = barrier_func(v_new)

                # Armijo condition
                if f_new <= f_current + alpha * s * np.dot(grad.T, delta):
                    break

            s *= beta

        # Update variables
        v0 += s * delta
        v_seq.append(v0.copy())

    return v_seq

def barr_method(Q, p, A, b, v0, eps, mu=10):
    """
    Implements the barrier method to solve a Quadratic Program (QP).

    Parameters:
        Q (ndarray): Quadratic term in the objective function (symmetric matrix).
        p (ndarray): Linear term in the objective function.
        A (ndarray): Constraint matrix.
        b (ndarray): Constraint vector.
        v0 (ndarray): Initial feasible point.
        eps (float): Precision criterion for stopping.
        mu (float): Scaling factor for barrier parameter (default: 10).

    Returns:
        v_seq (list): Sequence of iterates from each centering step.
        prec_criterion (list): Sequence of precision criteria values.
    """
    t = 1.0 # Initial barrier parameter
    m = A.shape[0] # Number of constraints
    v_seq = []
    prec_criterion = []

    while True:
        # Perform centering step
        cent_seq = centering_step(Q, p, A, b, t, v0, eps)
        v_seq.append(cent_seq)

        # Update v0 to the last iterate from centering step
        v0 = cent_seq[-1]

        # Compute precision criterion
        prec = m / t
        prec_criterion.append(prec)

        # Stopping criterion
        if prec < eps:
            return v_seq, prec_criterion

        # Update barrier parameter
        t *= mu

def generate_problem_data(n, d, lambd, seed=42):
    """
    Generate random problem data for testing.

    Parameters:
        n (int): Number of samples.
        d (int): Number of features.
        lambd (float): Regularization parameter.
        seed (int): Random seed for reproducibility.

    Returns:
        Tuple of (Q, p, A, b, y)
    """
    np.random.seed(seed)
    X = np.random.randn(n, d)
    y = np.random.randn(n)
    A = np.concatenate([X, -X], axis=1).T
    Q = np.eye(n) * 0.5
    p = -y
    b = lambd * np.ones(2 * d)
    return Q, p, A, b, y

def compute_objective(Q, p, v):
    """
    Compute the objective function value.

    Parameters:
        Q (ndarray): Quadratic term.
        p (ndarray): Linear term.
        v (ndarray): Variable vector.

    Returns:
        float: Objective function value.
    """
    return np.dot(v.T, np.dot(Q, v)) + np.dot(p.T, v)

def analyze_barrier_method(mus=[2, 15, 50, 100, 250, 500, 1000]):
    """
    Analyze barrier method performance for different mu values.

    Parameters:
        mus (list): List of barrier method parameter values.
    """
    n, d = 40, 50
    lambd = 10
    eps = 1e-8

    # Generate problem data
    Q, p, A, b, _ = generate_problem_data(n, d, lambd)
    v0 = np.zeros(n)

    # Store results
    precisions_all = []
    gaps_all = []

    # Run for different mu values
    for mu in mus:
        # Run barrier method
        v_seq, precisions = barr_method(Q, p, A, b, v0, eps, mu)
        precisions_all.append(precisions)

        # Flatten iterates and compute reference optimum
        all_iterates = [v for seq in v_seq for v in seq]
        v_opt = v_seq[-1][-1]
        f_opt = compute_objective(Q, p, v_opt)

        # Compute gaps
        gaps = [compute_objective(Q, p, v) - f_opt for v in all_iterates[1:-1]]
        gaps_all.append(gaps)

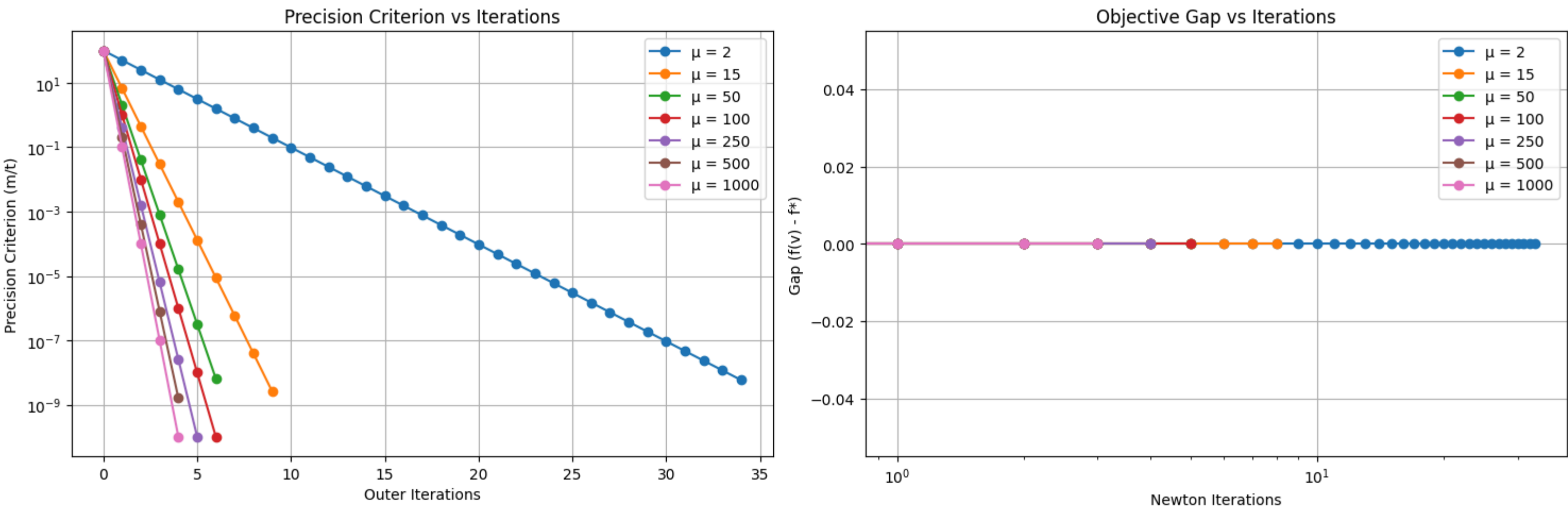
    # Plotting
    plt.figure(figsize=(15, 5))

    # Precision Criterion Plot
    plt.subplot(1, 2, 1)
    for precisions, mu in zip(precisions_all, mus):
        plt.plot(precisions, label=f'μ = {mu}', marker='o')
    plt.xlabel('Outer Iterations')
    plt.ylabel('Precision Criterion (m/t)')
    plt.title('Precision Criterion vs Iterations')
    plt.legend()
    plt.yscale('log')
    plt.grid(True)

    # Gap Plot
    plt.subplot(1, 2, 2)
    for gaps, mu in zip(gaps_all, mus):
        plt.semilogx(gaps, label=f'μ = {mu}', marker='o')
    plt.xlabel('Newton Iterations')
    plt.ylabel('Gap (f(v) - f*)')
    plt.title('Objective Gap vs Iterations')
    plt.legend()
    plt.grid(True)

    plt.tight_layout()
    plt.show()

# Run the analysis
if __name__ == "__main__":
    analyze_barrier_method()
```



Precision Criterion vs Iterations:

As the value of μ increases, the barrier method converges faster, as indicated by the faster decrease in the precision criterion. However, the precision criterion decreases more slowly for smaller values of μ , suggesting a more accurate solution.

Objective Gap vs Iterations:

For smaller values of μ , the objective function gap decreases more slowly, but it eventually reaches a lower minimum value compared to larger values of μ . This suggests that smaller values of μ , such as 2 or 15, can provide more accurate solutions, although at the cost of slower convergence.